

Self learning.

1

EXISTS vs IN Performance Comparison

- IN: Evaluates the subquery first, builds an in-memory result list, and then checks the outer query against it. It is best when the subquery dataset is small.
 - EXISTS: Stops scanning as soon as it finds a single matching row . It is highly efficient when the outer query dataset is small or the subquery matches rows early.
-

2.

```
RD3-98849-sakshi@localhost> DELIMITER $$
RD3-98849-sakshi@localhost>
RD3-98849-sakshi@localhost> CREATE TRIGGER update_cart_price
-> AFTER UPDATE ON products
-> FOR EACH ROW
-> BEGIN
->     IF NEW.price <> OLD.price THEN
->
->         UPDATE cart
->         SET price = NEW.price
->         WHERE product_id = NEW.id;
->
->     END IF;
-> END$$
Query OK, 0 rows affected (0.241 sec)

RD3-98849-sakshi@localhost>
RD3-98849-sakshi@localhost> DELIMITER ;
RD3-98849-sakshi@localhost> AFTER UPDATE ON products
-> ^C
RD3-98849-sakshi@localhost> UPDATE products
-> SET price = 55000
-> WHERE id = 1;
Query OK, 0 rows affected (0.157 sec)
Rows matched: 0  Changed: 0  Warnings: 0

RD3-98849-sakshi@localhost> SELECT * FROM products;
Empty set (0.040 sec)

RD3-98849-sakshi@localhost> SELECT * FROM cart;
Empty set (0.010 sec)

RD3-98849-sakshi@localhost> INSERT INTO users(username, email)
-> VALUES('sakshi', 'sakshi@gmail.com');
Query OK, 1 row affected (0.161 sec)
```

```
RD3-98849-sakshi@localhost> SELECT * FROM cart;
+-----+-----+-----+-----+-----+
| id | user_id | product_id | price | quantity |
+-----+-----+-----+-----+-----+
| 1 | 1 | 1 | 55000.00 | 1 |
+-----+-----+-----+-----+-----+
1 row in set (0.007 sec)
```

Q.3.

1,2

```
db.createCollection("emp")
{ ok: 1 }
db.createCollection("dept")
{ ok: 1 }
```

3.

```
> db.emp.updateMany(
  {deptno: 10},
  {$inc: {sal: 100}}
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 3,
  upsertedCount: 0
}
```

4.

```
> db.emp.find(
  {sal: {$gt: 1000}},
  {_id: 0, ename: 1, job: 1, deptno: 1, sal: 1}
).sort(
  {deptno: 1, job: 1, sal: -1}
)
< {
  ename: 'MILLER',
  job: 'CLERK',
  sal: 1400,
  deptno: 10
}
{
  ename: 'CLARK',
  job: 'MANAGER',
  sal: 2550,
  deptno: 10
}
{
  ename: 'KING',
  job: 'PRESIDENT',
  sal: 5100,
  deptno: 10
}
{
  ename: 'SCOTT',
  job: 'ANALYST',
  sal: 3000,
  deptno: 10
}
```

```
db.dept.deleteOne({deptno: 40})
{
  acknowledged: true,
  deletedCount: 1
}
```

5.

Q2

1

```
> db.emp.aggregate([
  {
    $match: {
      job: {$ne: "PRESIDENT"}
    }
  },
  {
    $group: {
      _id: "$job",
      total_sal: {$sum: "$sal"},
      max_sal: {$max: "$sal"},
      min_sal: {$min: "$sal"},
      avg_sal: {$avg: "$sal"}
    }
  }
])
< {
  _id: 'ANALYST',
  total_sal: 6000,
  max_sal: 3000,
  min_sal: 3000,
  avg_sal: 3000
}
{
  _id: 'SALESMAN',
  total_sal: 5600,
  max_sal: 1600,
  min_sal: 1250,
```

```

db.emp.aggregate([
  {
    $lookup: {
      from: "dept",
      localField: "deptno",
      foreignField: "deptno",
      as: "dept_info"
    }
  },
  {
    $unwind: "$dept_info"
  },
  {
    $project: {
      _id: 0,
      ename: 1,
      dname: "$dept_info.dname",
      income: {
        $add: [
          "$sal",
          ${ifNull: ["$comm", 0]}
        ]
      }
    }
  },
  {
    $sort: {
      income: -1
    }
  }
])
{
  ename: 'KING',
  dname: 'ACCOUNTING',
  income: 5100
}
{
  ename: 'SCOTT',
  dname: 'RESEARCH',
  income: 3000
}
}

```

2.

Group By All Answers.

Accepted 24 / 24 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:25

₹583.25/mo Save over 66%

Unlock the Full LeetCode... Company problems, Ask Leet, and ...

Runtime

697 ms Beats 59.47%



```
1 # Write your MySQL query statement below
2 SELECT
3     user_id,
4     MAX(time_stamp) AS last_stamp
5 FROM
6     Logins
7 WHERE
8     YEAR(time_stamp) = 2020
9 GROUP BY
10     user_id;
```

Accepted 12 / 12 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:24

₹583.25/mo Save over 66%

Unlock the Full LeetCode... Company problems, Ask Leet, and ...

Runtime

407 ms Beats 50.86%



```
1 # Write your MySQL query statement
2 SELECT actor_id, director_id
3 FROM ActorDirector
4 GROUP BY actor_id, director_id
5 HAVING COUNT(*) >= 3;
```

DescriptionAccepted × EditorialSolution

← All Submissions

Accepted22 / 22 testcases passed

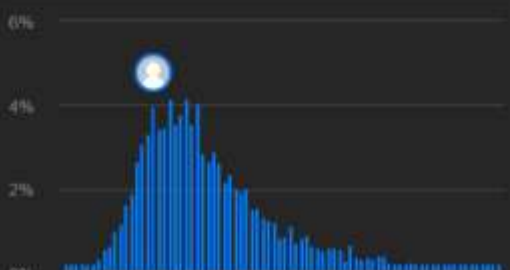
sakshi_sanjay_patil submitted at Sep 15, 2026 17:23

₹583.25/mo
Save over 56%

Unlock the Full LeetCode...
Company problems, Ask Leet, and ...

Runtime

536 ms | Beats 82.36%



Note ×CodeTestcaseTest Result

MySQLAuto

```
1 # Write your MySQL query statement below
2 SELECT
3     event_day AS day,
4     emp_id,
5     SUM(out_time - in_time) AS total_time
6 FROM
7     Employees
8 GROUP BY
9     event_day,
10    emp_id;
```

Saved

DescriptionAccepted × EditorialSolution

← All Submissions

Accepted22 / 22 testcases passed

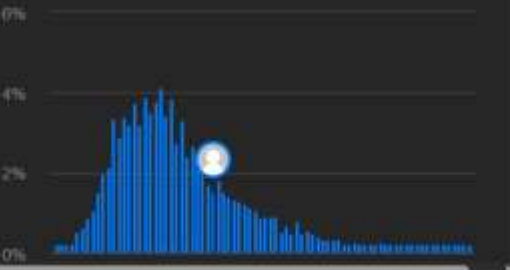
sakshi_sanjay_patil submitted at Sep 15, 2026 17:21

₹583.25/mo
Save over 56%

Unlock the Full LeetCode...
Company problems, Ask Leet, and ...

Runtime

677 ms | Beats 29.21%



Note ×CodeTestcaseTest Result

MySQLAuto

```
1 # Write your MySQL query statement below
2 SELECT
3     date_id,
4     make_name,
5     COUNT(DISTINCT lead_id) AS unique_leads,
6     COUNT(DISTINCT partner_id) AS unique_partners
7 FROM
8     DailySales
9 GROUP BY
10    date_id,
11    make_name;
```

Saved

DescriptionAccepted × EditorialSolution

← All Submissions

Accepted12 / 12 testcases passed
sakshi_sanjay_patil submitted at Sep 15, 2026 17:20

₹583.25/mo
Save over 58%

Unlock the Full LeetCode...
Company problems, Ask Leet, and ...

Runtime

489 ms | Beats 85.51%

Note × CodeTestcaseTest Result

MySQLAuto

```
1 # Write your MySQL query statement below
2 SELECT player_id, MIN(event_date) AS first_login
3 FROM Activity
4 GROUP BY player_id;
5
```

Saved

DescriptionAccepted × EditorialSolution

← All Submissions

Accepted18 / 18 testcases passed
sakshi_sanjay_patil submitted at Sep 15, 2026 17:18

₹583.25/mo
Save over 58%

Unlock the Full LeetCode...
Company problems, Ask Leet, and ...

Runtime

442 ms | Beats 77.01%

Note × CodeTestcaseTest Result

MySQLAuto

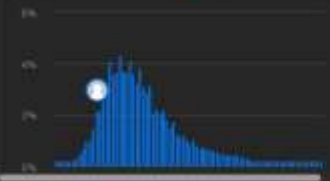
```
1 # Write your MySQL query statement below
2 SELECT MAX(num) AS num
3 FROM (
4     SELECT num
5     FROM MyNumbers
6     GROUP BY num
7     HAVING COUNT(num) = 1
8 ) AS unique_numbers;
9
```

Saved

Accepted 11 / 19 testcases passed
sakshi_sanjay_patil submitted at Sep 15, 2026 17:17

Unlock the Full LeetCode...
Company problems, Ask Leet, and ...

Runtime
446 ms Beats 93.24%



```
1 # Write your MySQL query statement below
2 SELECT customer_number
3 FROM Orders
4 GROUP BY customer_number
5 ORDER BY COUNT(order_number) DESC
6 LIMIT 1;
```

Accepted 13 / 13 testcases passed
sakshi_sanjay_patil submitted at Sep 15, 2026 17:27

Unlock the Full LeetCode...
Company problems, Ask Leet, and ...

Runtime
403 ms Beats 52.41%



```
1 # Write your MySQL query statement below
2 SELECT
3     query_name,
4     ROUND(AVG(rating / position), 2) AS quality,
5     ROUND(AVG(rating < 3) * 100, 2) AS poor_query_percentage
6 FROM
7     Queries
8 WHERE
9     query_name IS NOT NULL
10 GROUP BY
11     query_name;
```

Joins

1

Description

Accepted

Editorial

Solutions

All Submissions

Accepted

25 / 25 testcases passed

sakshi_sanjay_patil

submitted at Sep 15, 2026 17:33

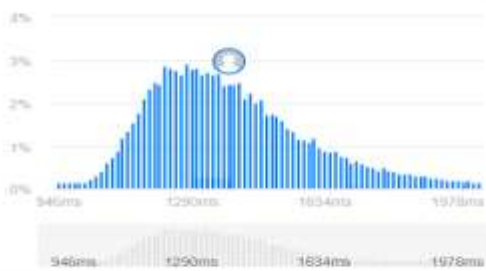
₹583.25

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert ...

Runtime

1399 ms | Beats 45.51%



Code: MySQL

1 # Write your MySQL query statement below

Code

MySQL

Auto

1 # Write your MySQL query statement below

2 SELECT eu.unique_id, e.name

3 FROM Employees e

4 LEFT JOIN EmployeeUNI eu ON e.id = eu.id;

5

Testcase

Test Result

Accepted

Runtime: 1106 ms

Case 1

Input

Employees =

id	name
1	Alice
7	Bob
11	Meir
90	Winston
3	Jonathan

2.

All Submissions

Accepted

Editorial

Solutions

All Submissions

Accepted

13 / 13 testcases passed

sakshi_sanjay_patil

submitted at Sep 15, 2026 17:34

₹583.25

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert ...

Runtime

1243 ms | Beats 82.12%



Code: MySQL

1 # Write your MySQL query statement below

Code

MySQL

Auto

1 # Write your MySQL query statement below

2 SELECT p.product_name, s.year, s.price

3 FROM Sales s

4 JOIN Product p

5 ON s.product_id = p.product_id;

6

Testcase

Test Result

Accepted

Runtime: 93 ms

Case 1

Input

Sales =

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Product =

3

Accepted 21 / 21 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:35

Unlock the Full LeetCode Experience

```

1 # Write your MySQL query statement below
2 SELECT e.name, b.bonus
3 FROM Employee e
4 LEFT JOIN Bonus b ON e.empId = b.empId
5 WHERE b.bonus < 1000 OR b.bonus IS NULL;
6

```

4.

Accepted 15 / 15 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:37

₹583.25/mo Save over 50%

Unlock the Full LeetCode Experience →
Company problems, Ask Leet, and expert ...

Runtime

1141 ms Beats 50.39%

775ms 1078ms 1376ms 1675ms

```

1 # Write your MySQL query statement below
2 SELECT
3     s.student_id,
4     s.student_name,
5     sub.subject_name,
6     COUNT(e.subject_name) AS attended_exams
7 FROM Students s
8 CROSS JOIN Subjects sub
9 LEFT OUTER JOIN Examinations e
10     ON s.student_id = e.student_id
11     AND sub.subject_name = e.subject_name
12 GROUP BY
13     s.student_id,
14     s.student_name,
15     sub.subject_name
16 ORDER BY
17     s.student_id,
18     sub.subject_name;
19

```

5.

Accepted 11 / 11 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:38

₹583.25/mo Save over 50%

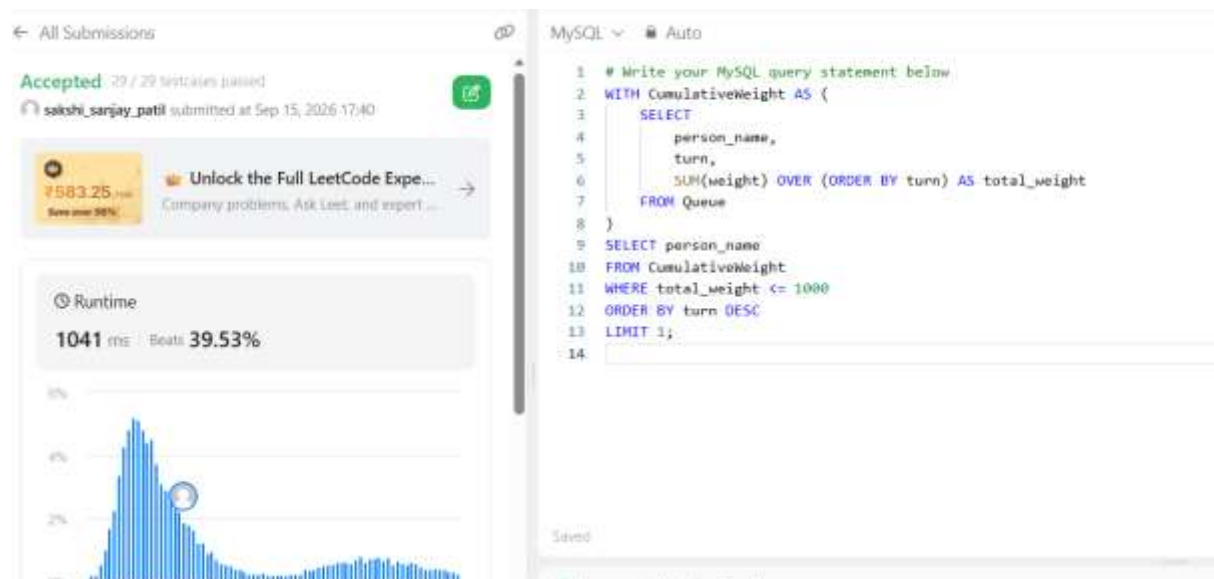
Unlock the Full LeetCode Experience →
Company problems, Ask Leet, and expert ...

```

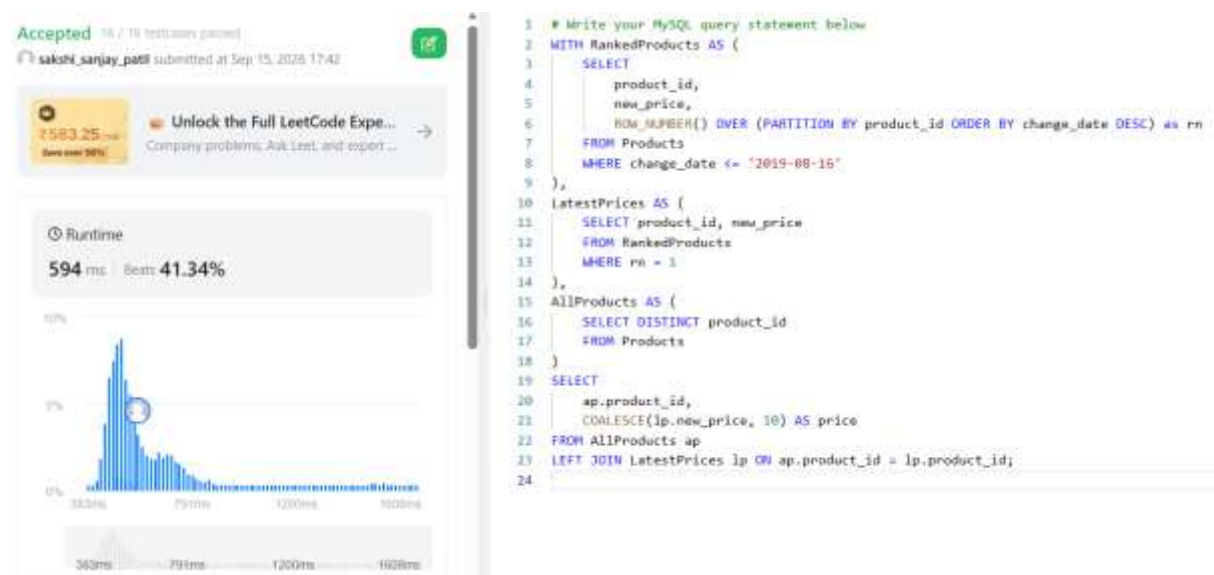
1 # Write your MySQL query statement below
2 SELECT e1.name
3 FROM Employee AS e1
4 JOIN (
5     SELECT managerId
6     FROM Employee
7     GROUP BY managerId
8     HAVING COUNT(managerId) >= 5
9 ) AS e2
10 ON e1.id = e2.managerId;

```

8.

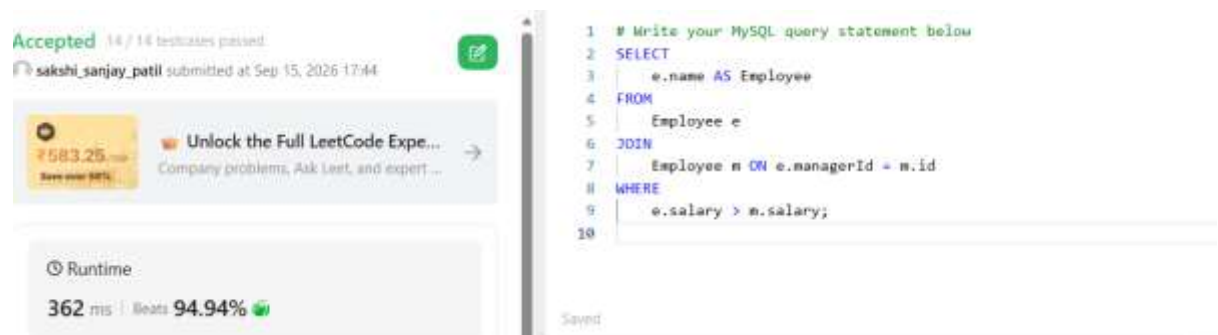


9.



Subquery Problem

1.

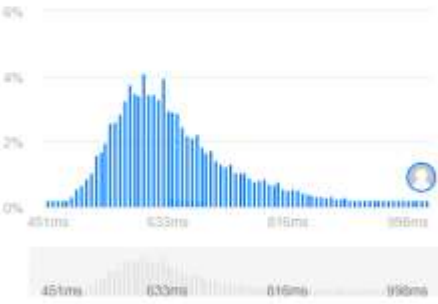


2.

Accepted 12 / 12 testcases passed
sakshi_sanjay_patil submitted at Sep 15, 2026 17:43

2583.25ms
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert...

Runtime
1747 ms Beats 5.00%



Code: MySQL

```
1 # Write your MySQL query statement below
2 SELECT
3     c.name AS Customers
4 FROM
5     Customers c
6 LEFT JOIN
7     Orders o ON c.id = o.customerId
8 WHERE
9     o.id IS NULL;
```

Accepted Runtime: 116 ms

Case 1

3.

All Submissions

Accepted 15 / 15 testcases passed
sakshi_sanjay_patil submitted at Sep 15, 2026 17:46

2583.25ms
Unlock the Full LeetCode Experience
Company problems, Ask Leet, and expert...

Runtime
687 ms Beats 11.52%



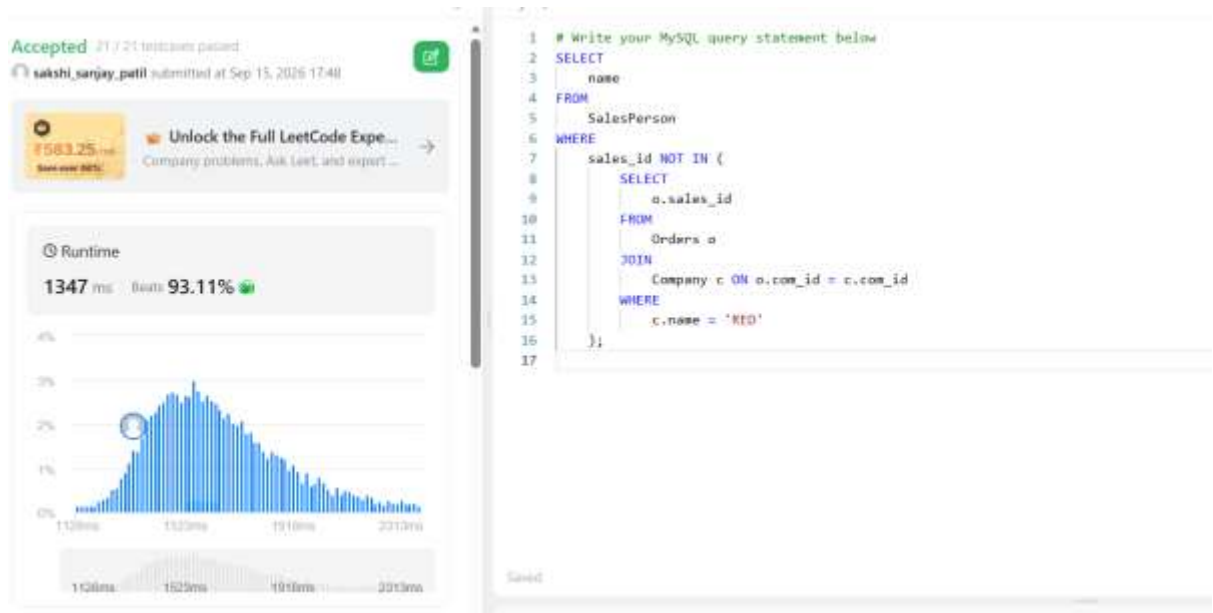
Code: MySQL

```
1 # Write your MySQL query statement below
2 SELECT
3     w1.id
4 FROM
5     Weather w1
6 JOIN
7     Weather w2 ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
8 WHERE
9     w1.temperature > w2.temperature;
```

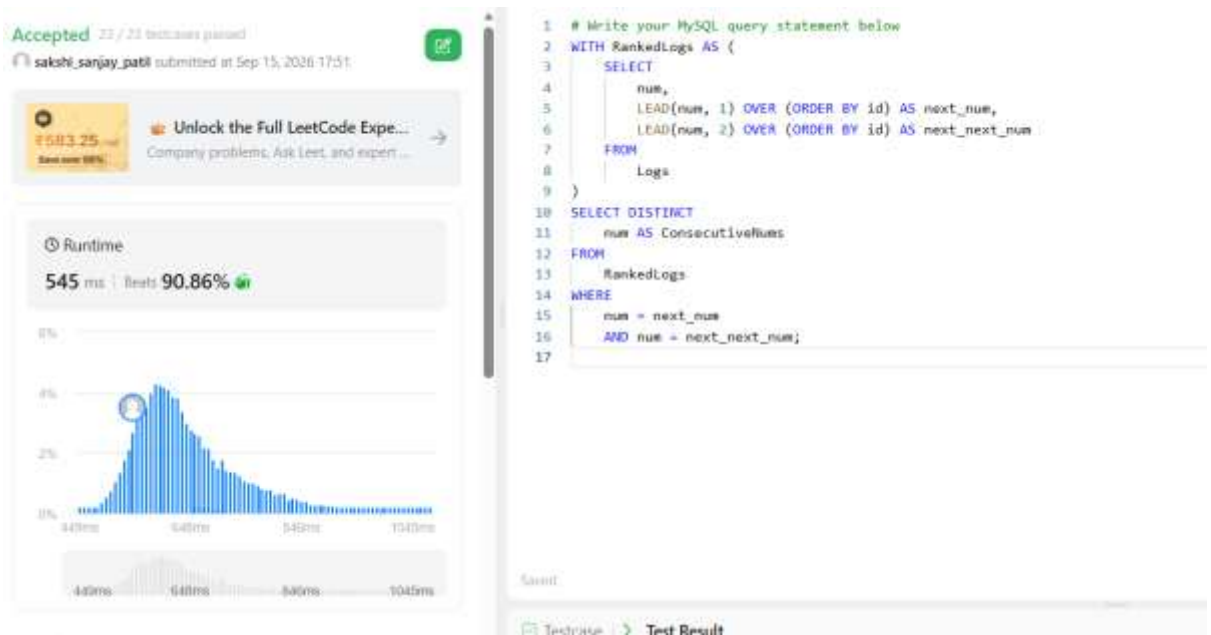
Accepted Runtime: 88 ms

Case 1

4.



5.



6.

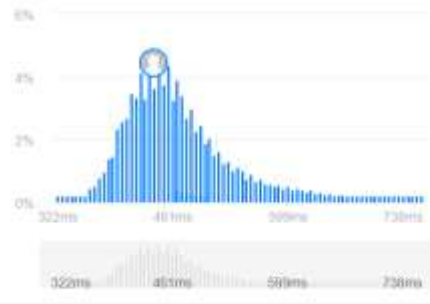
Accepted 0 / 0 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:52




 Unlock the Full LeetCode Experience...
 Company problems, Ask Leet, and expert...

Runtime
439 ms Beats **66.90%**



Code MySQL

1 # Write your MySQL query statement below

2 SELECT

3 p.firstName,

4 p.lastName,

5 a.city,

6 a.state

7 FROM

8 Person p

9 LEFT JOIN

10 Address a ON p.personId = a.personId;

11

Saved

Testcase Test Result

7.

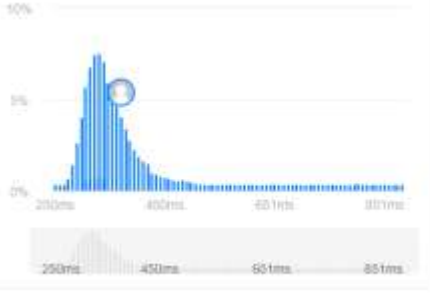
Accepted 11 / 11 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:53




 Unlock the Full LeetCode Experience...
 Company problems, Ask Leet, and expert...

Runtime
370 ms Beats **38.70%**



Code MySQL

1 # Write your MySQL query statement below

2 SELECT

3 score,

4 DENSE_RANK() OVER (ORDER BY score DESC) AS 'rank'

5 FROM

6 Scores;

7

Saved

Testcase Test Result

8.

All Submissions

MySQL Auto

Accepted 11 / 11 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:54

583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert ...

Runtime

357 ms Beats 77.44%

Code MySQL

Testcase Test Result

```

1 # Write your MySQL query statement below
2 SELECT e1.name
3 FROM Employee AS e1
4 JOIN (
5     SELECT managerId
6     FROM Employee
7     GROUP BY managerId
8     HAVING COUNT(managerId) >= 5
9 ) AS e2
10 ON e1.id = e2.managerId;
11

```

9.

Description Accepted Editorial Solutions

All Submissions

Accepted 22 / 22 testcases passed

sakshi_sanjay_patil submitted at Sep 15, 2026 17:55

583.25ms

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert ...

Runtime

824 ms Beats 83.06%

Code MySQL Auto

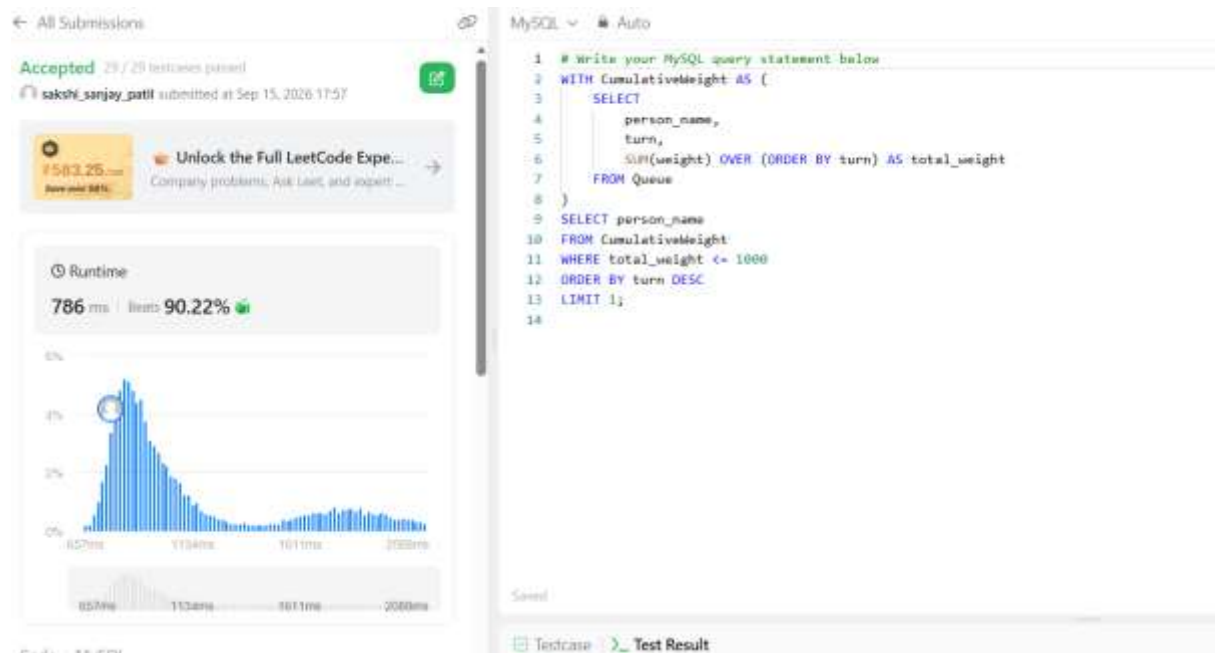
Testcase Test Result

```

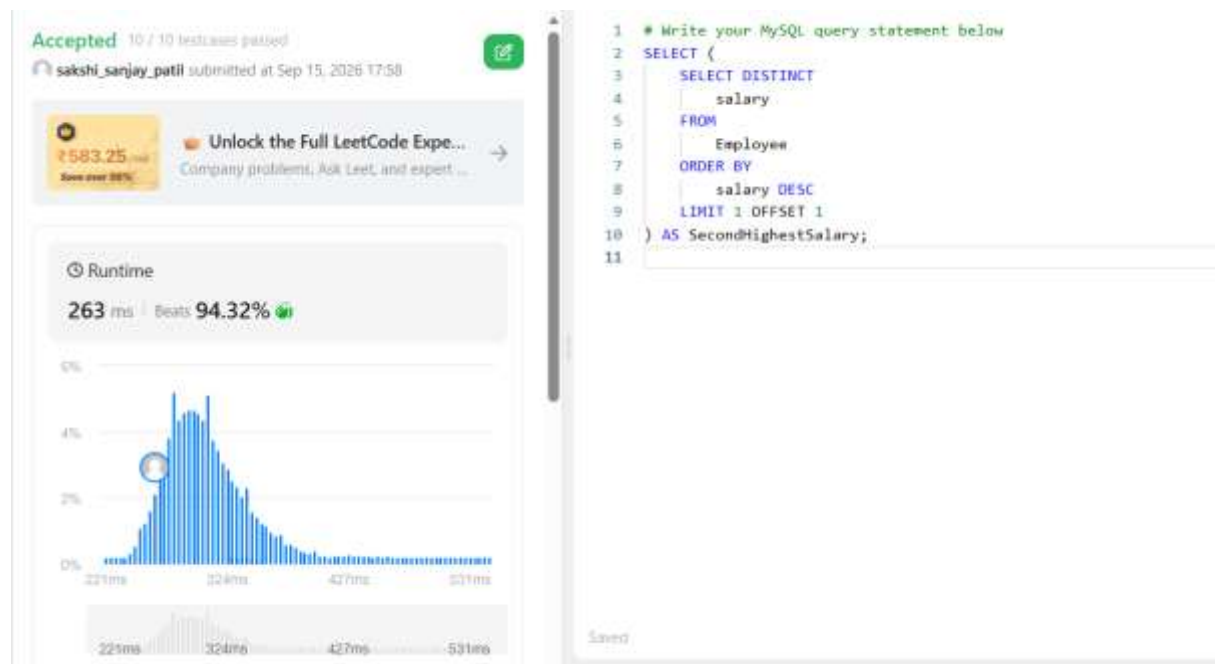
1 # Write your MySQL query statement below
2 DELETE p1
3 FROM
4     Person p1
5 JOIN
6     Person p2 ON p1.email = p2.email
7 WHERE
8     p1.id > p2.id;
9

```

10.



11.



Written Exercises

1. Explain Conceptual, Logical and Physical data models for the tables: emp, dept, meeting, address

Conceptual Model: Focuses on **business rules**. Identifies high-level entities (emp, dept, meeting, address) and their relationships without any technical details.

Logical Model: Focuses on **data structure**. Defines specific column names, Primary Keys (PK), and Foreign Keys (FK) independent of the database software.

Physical Model: Focuses on **database implementation**. Specifies vendor-specific data types (INT, VARCHAR), nullability (NOT NULL), and indexes for the database engine.

2. Understanding locks: Shared locks (Read lock), Exclusive locks (Write lock), Row-level vs Table-level locks

- **Shared Lock (Read):** Allows multiple users to read a row simultaneously but blocks any modifications.
- **Exclusive Lock (Write):** Allows only one user to update a row, blocking all other reads and writes.
- **Row-level Lock:** Locks specific rows. Offers high concurrency but consumes more system memory.
- **Table-level Lock:** Locks the entire table. Restricts access for other users but uses minimal system memory.

Case Study

```
KD3-98849-sakshi@localhost> SELECT * FROM student;
```

student_id	name	email	mobile	rank_no	score
1	Amit	amit@gmail.com	9000000001	100	95.00
2	Sakshi	sakshi@gmail.com	9000000002	150	92.00
3	Rahul	rahul@gmail.com	9000000003	200	90.00
4	Priya	priya@gmail.com	9000000004	250	88.00
5	Neha	neha@gmail.com	9000000005	300	85.00

```
5 rows in set (0.013 sec)
```

```
KD3-98849-sakshi@localhost> SELECT * FROM center;
```

center_id	center_name	city
1	Sunbeam Karad	Karad
2	Sunbeam Pune	Pune
3	CDAC Mumbai	Mumbai

```
3 rows in set (0.010 sec)
```

```
KD3-98849-sakshi@localhost> SELECT * FROM course;
```

course_id	course_name	center_id	total_seats	available_seats
1	DAC	1	3	0
2	DAC	2	2	0
3	DBDA	2	2	2
4	DAC	3	2	2

```
4 rows in set (0.008 sec)
```

```
KD3-98849-sakshi@localhost> SELECT * FROM application;
```

application_id	student_id	application_date	status
1	1	2026-09-15	APPLIED
2	2	2026-09-15	APPLIED
3	3	2026-09-15	APPLIED

```

KD3-98849-sakshi@localhost> CREATE PROCEDURE round1_allocation()
-> BEGIN
->
->     DECLARE done INT DEFAULT 0;
->
->     DECLARE v_application_id INT;
->     DECLARE v_course_id INT;
->     DECLARE v_pref_no INT;
->     DECLARE v_seats INT;
->
->     DECLARE cur CURSOR FOR
->         SELECT a.application_id
->         FROM application a
->         JOIN student s
->         ON a.student_id = s.student_id
->         ORDER BY s.rank_no;
->
->     DECLARE CONTINUE HANDLER FOR NOT FOUND
->         SET done = 1;
->
->     OPEN cur;
->
->     student_loop: LOOP
->
->         FETCH cur INTO v_application_id;
->
->         IF done = 1 THEN
->             LEAVE student_loop;
->         END IF;
->
->         SET v_pref_no = 1;
->
->         preference_loop: LOOP
->
->             SELECT course_id
->             INTO v_course_id
->             FROM preference
->             WHERE application_id = v_application_id
->             AND preference_no = v_pref_no;
->
->             SELECT available_seats
->             INTO v_seats
->             FROM course
->             WHERE course_id = v_course_id;
->
->             IF v_seats > 0 THEN
->
->                 INSERT INTO allocation
->                 (application_id, course_id, round_no)
->                 VALUES
->                 (v_application_id, v_course_id, 1);
->
->                 UPDATE course
->                 SET available_seats = available_seats - 1
->                 WHERE course_id = v_course_id;
->
->                 LEAVE preference_loop;
->
->             END IF;
->
->             SET v_pref_no = v_pref_no + 1;
->
->         END LOOP;
->
->     END LOOP;
->
->     CLOSE cur;

```

```
KD3-98849-sakshi@localhost> DELIMITER ;
KD3-98849-sakshi@localhost> CALL round1_allocation();
Query OK, 0 rows affected (0.562 sec)
```

```
KD3-98849-sakshi@localhost> SELECT * FROM allocation;
```

allocation_id	application_id	course_id	round_no	allocation_date
1	1	1	1	2026-09-15
2	2	1	1	2026-09-15
3	3	1	1	2026-09-15
4	4	2	1	2026-09-15
5	5	2	1	2026-09-15

```
5 rows in set (0.015 sec)
```

```

KD3-98849-sakshi@localhost>
KD3-98849-sakshi@localhost> CREATE PROCEDURE round2_allocation()
-> BEGIN
->
->     DECLARE done INT DEFAULT 0;
->     DECLARE v_application_id INT;
->     DECLARE v_course_id INT;
->     DECLARE v_pref_no INT;
->     DECLARE v_max_pref INT;
->     DECLARE v_seats INT;
->
->     DECLARE cur CURSOR FOR
->         SELECT a.application_id
->         FROM application a
->         JOIN student s
->         ON a.student_id = s.student_id
->         WHERE NOT EXISTS (
->             SELECT 1
->             FROM allocation al
->             WHERE al.application_id = a.application_id
->         )
->         ORDER BY s.rank_no;
->
->     DECLARE CONTINUE HANDLER FOR NOT FOUND
->         SET done = 1;
->
->     OPEN cur;
->
->     student_loop: LOOP
->
->         FETCH cur INTO v_application_id;
->
->         IF done = 1 THEN
->             LEAVE student_loop;
->         END IF;
->
->         SET v_pref_no = 1;
->
->         SELECT COUNT(*)
->         INTO v_max_pref
->         FROM preference
->         WHERE application_id = v_application_id;
->
->         preference_loop: LOOP
->
->             IF v_pref_no > v_max_pref THEN
->                 LEAVE preference_loop;
->             END IF;
->
->             SELECT course_id
->             INTO v_course_id
->             FROM preference
->             WHERE application_id = v_application_id
->             AND preference_no = v_pref_no;
->
->             SELECT available_seats

```

```

->         SELECT available_seats
->         INTO v_seats
->         FROM course
->         WHERE course_id = v_course_id;
->
->         IF v_seats > 0 THEN
->
->             INSERT INTO allocation
->             (application_id, course_id, round_no)
->             VALUES
->             (v_application_id, v_course_id, 2);
->
->             UPDATE course
->             SET available_seats = available_seats - 1
->             WHERE course_id = v_course_id;
->
->             LEAVE preference_loop;
->
->         END IF;
->
->         SET v_pref_no = v_pref_no + 1;
->
->     END LOOP;
->
-> END LOOP;
->
-> CLOSE cur;
->
-> END$$
query OK, 0 rows affected (0.074 sec)

D3-98849-sakshi@localhost>
D3-98849-sakshi@localhost> DELIMITER ;
D3-98849-sakshi@localhost> CALL round2_allocation();
query OK, 0 rows affected (0.062 sec)

D3-98849-sakshi@localhost> SELECT * FROM student;
+-----+-----+-----+-----+-----+
student_id | name  | email                | mobile      | rank_no | score |
+-----+-----+-----+-----+-----+
1 | Amit  | amit@gmail.com       | 9000000001  | 100     | 95.00 |
2 | Sakshi | sakshi@gmail.com     | 9000000002  | 150     | 92.00 |
3 | Rahul | rahul@gmail.com      | 9000000003  | 200     | 90.00 |
4 | Priya | priya@gmail.com      | 9000000004  | 250     | 88.00 |
5 | Neha  | neha@gmail.com       | 9000000005  | 300     | 85.00 |
+-----+-----+-----+-----+-----+
rows in set (0.013 sec)

D3-98849-sakshi@localhost> SELECT * FROM center;
+-----+-----+-----+
center_id | center_name | city |
+-----+-----+-----+
1 | Sunbeam Karad | Karad |
2 | Sunbeam Pune | Pune |
3 | CDAC Mumbai | Mumbai |
+-----+-----+-----+

```

```
KD3-98849-sakshi@localhost> SELECT * FROM course;
```

course_id	course_name	center_id	total_seats	available_seats
1	DAC	1	3	0
2	DAC	2	2	0
3	DBDA	2	2	2
4	DAC	3	2	2

```
4 rows in set (0.008 sec)
```

```
KD3-98849-sakshi@localhost> SELECT * FROM application;
```

application_id	student_id	application_date	status
1	1	2026-09-15	APPLIED
2	2	2026-09-15	APPLIED
3	3	2026-09-15	APPLIED
4	4	2026-09-15	APPLIED
5	5	2026-09-15	APPLIED

```
5 rows in set (0.004 sec)
```

```
KD3-98849-sakshi@localhost> SELECT * FROM preference;
```

application_id	preference_no	course_id
1	1	1
2	1	1
3	1	1
5	1	1
1	2	2
2	2	2
4	1	2
5	2	2
2	3	3
3	2	3
4	2	3

```
11 rows in set (0.014 sec)
```

```
KD3-98849-sakshi@localhost> SELECT * FROM allocation;
```

allocation_id	application_id	course_id	round_no	allocation_date
1	1	1	1	2026-09-15
2	2	1	1	2026-09-15
3	3	1	1	2026-09-15
4	4	2	1	2026-09-15
5	5	2	1	2026-09-15

```
5 rows in set (0.009 sec)
```